

Lab 3: Architecture Justification

Kshitij G Shettigar / PES1UG24CS240 / CSE Sem-5 / Problem Statement 52: Community Tool & Equipment Library

We chose Layered Architecture for the Community Tool & Equipment Library System. Every loan passes through one chain (search, reserve, eligibility check, deposit, return, damage settlement), and every step reads and writes the same member, item and deposit records. Layering puts that whole chain in one Business layer over one Repository, so the rules linking the steps are never split across service boundaries.

Reason 1: the deposit rules need one consistent record. Checkout is refused while a member has overdue items or unpaid fees (FR-004). A return deducts a Rs.500 damage charge and a Rs.120 late fee from the Rs.2000 deposit and releases the Rs.1380 balance (FR-005). Both decisions read loan, fee and deposit data together. Loan Manager calls IEligibility and IPayment in-process and writes through a single IRepository, so a settlement never sees a half-updated deposit. Splitting these into microservices would create exactly the data-consistency problem the handout lists as that style's main weakness.

Reason 2: it fits who runs the library. The system serves one neighbourhood and is run by a single custodian, with no team to operate a fleet of independently deployed services. The layered system ships as one unit, yet each layer stays replaceable: changing payment provider touches only Payment Service behind IPayment, and nothing above it changes.

Security advantage: every request crosses a guarded boundary. The Presentation layer has no path to data except through Business-layer interfaces. Depot locker locations are returned only after Member Account authenticates the caller through IAuth, and an unauthenticated request gets HTTP 401 (NFR-001). Card details go only to the external Payment Gateway: Payment Service receives and stores a tokenised reference through IGatewayAPI, so the Library Repository holds no card data to leak (NFR-002).

Performance benefit: the busiest path is short and isolated. Catalogue search is the most frequent request, and it touches only Catalogue & Reservation and the Repository, never Loan Manager or the payment path. Because all data access goes through one IRepository, the indexes on category, keyword and availability date are defined in one place and serve every search. This is how the design targets $p_{95} \leq 2\text{ s}$ at 200 concurrent users (NFR-001). Calls inside the Business layer are also in-process, which avoids the network hop every microservice call would add.

Alternatives considered. Microservices were rejected for the consistency reason above. Plain Client-Server is not rejected so much as absorbed: the Member Web App and Custodian Console are this system's clients. Layering adds the internal separation that a single undivided server lacks.